Task 1: Agent Definition

1.1 Project Description

What problem does your project solve? Traditional personal finance applications rely on rigid, keyword based rules to categorize transactions, which fail when processing vague or abbreviated vendor names. This project eliminates tedious manual data entry by intelligently interpreting and categorizing raw banking data.

Who are the target users? Individuals seeking to track budgets, categorize expenditures, and gain insights into financial habits without manual data entry or complex spreadsheets.

What can the agent do? The agent analyzes financial records, explains spending patterns, automatically categorizes transactions, detects unusual expenses, tracks subscriptions, and answers natural language financial queries.

Why is an AI agent appropriate for this problem? Raw bank transactions often contain unstructured abbreviations that require contextual reasoning to decipher, which a deterministic rule engine cannot handle effectively.

Which AI/LLM model(s) do you plan to use? A lightweight LLM API (such as Gemini 1.5 Flash or GPT 4o mini) optimized for text classification and structured JSON output.

How will the AI model interact with the rest of the software system? The AI interacts via an AgentController and AgentFacade. The system sanitizes database data, constructs prompt templates, executes the API call, and parses the returned JSON back into domain objects for the GUI/CLI to display.

1.2 Feature Specification

1. F01 CSV Transaction Import

- **Description:** Parses raw banking CSV files and stores them locally.
- **User Interaction:** User uploads a CSV via the GUI file picker or specifies a file path via the CLI.
- **Input:** Raw CSV file.
- **Output:** Standardized Transaction objects saved to the database.
- **AI Involvement:** Deterministic.
- **Expected Workflow:** System reads the file, maps columns (Date, Amount, Description), drops duplicates, and saves to the database.
- **Error/Alternative Cases:** Malformed CSV prompts a "Format Error" message.

2. F02 AI Automatic Categorization

- **Description:** Automatically assigns a financial category to imported transactions.
- **User Interaction:** User clicks "Categorize Transactions" in the GUI.
- **Input:** Uncategorized Transaction descriptions.
- **Output:** Categorized Transaction objects.
- **AI Involvement:** AI based.

- **Expected Workflow:** Agent processes descriptions in batches, evaluates context, and returns mapped categories via JSON.
- **Error/Alternative Cases:** If the LLM returns an invalid category, the system flags the transaction as "Uncategorized".

3. F03 Natural Language Financial Queries

- **Description:** Users can ask plain text questions about their finances.
- **User Interaction:** User types a query into the GUI chat interface.
- **Input:** Natural language string (e.g., "Did I overspend on coffee?").
- **Output:** Textual answer and relevant data metrics.
- **AI Involvement:** AI based.
- **Expected Workflow:** Agent converts query into a database search, retrieves transactions via tools, and generates a conversational response.
- **Error/Alternative Cases:** If data is missing, the AI requests clarification.

4. F04 Spending Trend & Anomaly Detection

- **Description:** Identifies unusual spikes in spending.
- **User Interaction:** Automatic notification on the GUI dashboard.
- **Input:** Historical transaction data.
- **Output:** Alert messages (e.g., "Dining expenses are 40% higher this week").
- **AI Involvement:** Hybrid.
- **Expected Workflow:** Deterministic math calculates rolling averages; AI formulates the context of the anomaly for the user.
- **Error/Alternative Cases:** Insufficient historical data results in the feature remaining inactive.

5. F05 Recurring Payment Detection

- **Description:** Identifies subscriptions and regular bills.
- **User Interaction:** Displayed in a "Subscriptions" tab in the GUI.
- **Input:** Categorized transaction history.
- **Output:** List of recurring expenses with predicted next charge dates.
- **AI Involvement:** Hybrid.
- **Expected Workflow:** Agent searches for identical amounts or matching vendor strings occurring at 30 day intervals.
- **Error/Alternative Cases:** Variable utility bills might be missed; user can manually flag them.

6. F06 Monthly Budget Creation

- **Description:** Allows users to set spending limits per category and tracks progress.
- **User Interaction:** User inputs budget limits via GUI forms.
- **Input:** Integer amounts per category.
- **Output:** Progress bars showing current spend vs. limit.
- **AI Involvement:** Deterministic.
- **Expected Workflow:** System aggregates categorized transactions and compares them to set limits.

- **Error/Alternative Cases:** Negative budget inputs are rejected.

7. F07 AI Monthly Financial Summary Generation

- **Description:** Generates a personalized written report of the month's financial health.
- **User Interaction:** User clicks "Generate Summary" at month end.
- **Input:** Month's aggregated budget and transaction data.
- **Output:** A formatted text summary with insights and saving suggestions.
- **AI Involvement:** AI based.
- **Expected Workflow:** Controller aggregates monthly data and passes it to the Agent, which drafts a comprehensive review.
- **Error/Alternative Cases:** API timeout results in a prompt to retry.

8. F08 Manual Category Override

- **Description:** Users can manually correct AI categorizations.
- **User Interaction:** Dropdown menu selection on a transaction row.
- **Input:** Transaction ID and new Category ID.
- **Output:** Updated database record.
- **AI Involvement:** Hybrid.
- **Expected Workflow:** Database updates the category. The Agent saves this correction to local memory to improve future categorizations.
- **Error/Alternative Cases:** Database write lock delays update.

9. F09 Financial Report Export

- **Description:** Exports filtered data as a formatted document.
- **User Interaction:** User selects date range and clicks "Export".
- **Input:** Date range constraints.
- **Output:** Downloadable CSV report.
- **AI Involvement:** Deterministic.
- **Expected Workflow:** Controller queries the database and formats the results into the requested file type.
- **Error/Alternative Cases:** Empty date range yields an empty file with a warning.

10. F10 CLI Batch Processing Engine

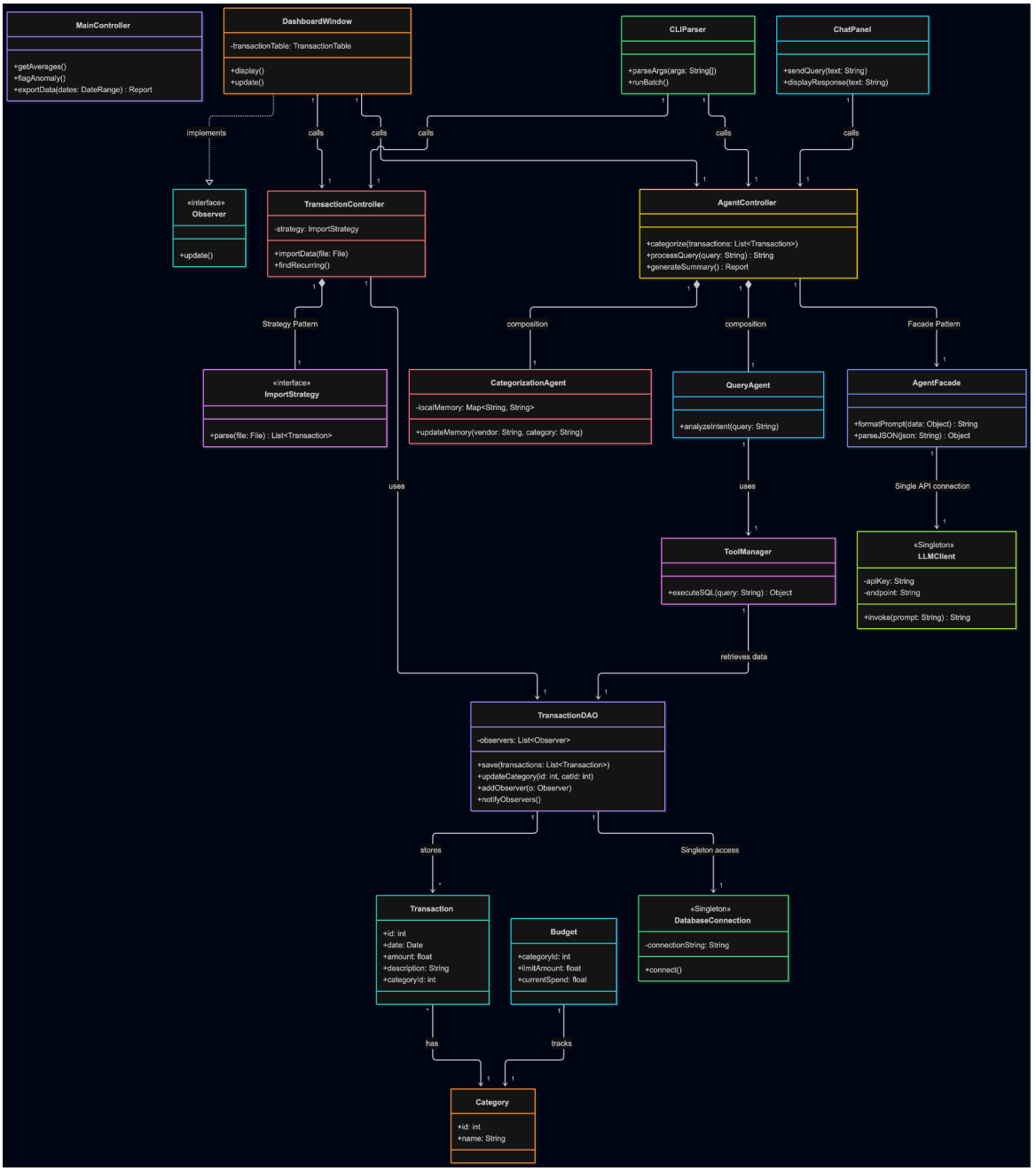
- **Description:** Allows users to access the major functionality of the agent via terminal commands.
- **User Interaction:** User runs terminal commands (e.g., finance agent import data.csv categorize).
- **Input:** CLI arguments.
- **Output:** Console logs and updated database.
- **AI Involvement:** Hybrid.
- **Expected Workflow:** CLI parser triggers the import, triggers the Agent categorization, and outputs a summary to stdout.
- **Error/Alternative Cases:** Invalid arguments trigger the CLI help menu.

Task 2: System Design UML

2.1 Class Diagram

(Textual representation of the required UML class diagram components)

- **Major Classes & Interfaces:** DashboardWindow, TransactionTable, ChatPanel, CLIParser, MainController, TransactionController, AgentController, Transaction, Category, Budget, Report, LLMClient, CategorizationAgent, QueryAgent, ToolManager, DatabaseConnection, TransactionDAO.
- **Attributes:** Transaction (id, date, amount, description, categoryId), Budget (categoryId, limitAmount, currentSpend).
- **Methods:** AgentController.categorize(), TransactionDAO.save(), LLMClient.invoke().
- **Relationships:** TransactionController depends on TransactionDAO; AgentController aggregates CategorizationAgent and QueryAgent; DashboardWindow observes TransactionDAO.



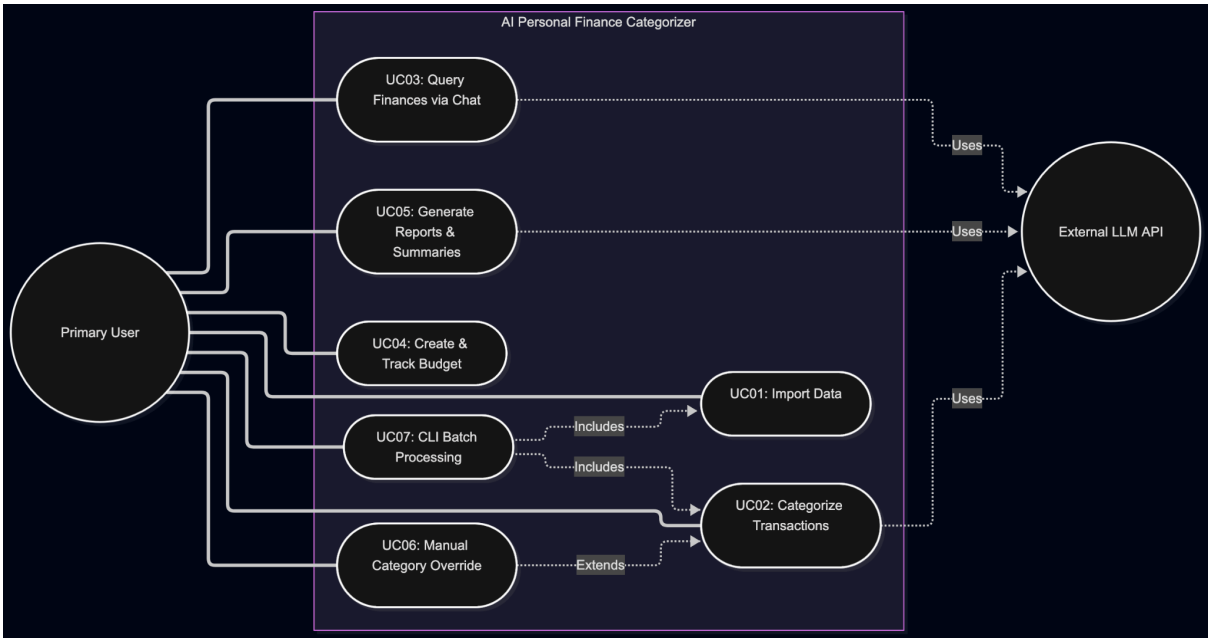
Design Patterns

1. **MVC (Model View Controller)**
    - **Problem Addressed:** Tightly coupling UI code with database and AI logic makes the application fragile.
    - **Participating Classes:** DashboardWindow (View), MainController (Controller), Transaction (Model).
    - **Role of Each Class:** View handles rendering, Controller mediates inputs, Model holds data.
    - **Why Appropriate:** Decouples user interfaces (both GUI and CLI) from core financial logic.
    - **Difficulty Without It:** Adding the CLI alongside the GUI would require duplicating database and AI interaction code.
2. **Singleton**
    - **Problem Addressed:** Multiple concurrent connections to the local database or external LLM API can cause locking errors.
    - **Participating Classes:** DatabaseConnection, LLMClient.
    - **Role of Each Class:** Provides a single, globally accessible instance of the resource connection.
    - **Why Appropriate:** Ensures strict governance over shared external resources.
    - **Difficulty Without It:** Application could crash due to concurrent write attempts.
3. **Strategy**
    - **Problem Addressed:** Parsing different bank CSV formats requires varied algorithms.
    - **Participating Classes:** ImportStrategy (Interface), TDCSVStrategy, RBCStrategy, TransactionController.
    - **Role of Each Class:** ImportStrategy defines the contract; concrete strategies implement bank specific parsing; TransactionController executes the selected strategy.
    - **Why Appropriate:** Allows dynamic swapping of parsing algorithms based on file type without altering the controller.
    - **Difficulty Without It:** The controller would become bloated with complex if/else logic.
4. **Observer**
    - **Problem Addressed:** The GUI needs to update immediately when the background AI categorizes 100 new transactions.
    - **Participating Classes:** TransactionDAO (Subject), DashboardWindow (Observer).
    - **Role of Each Class:** TransactionDAO broadcasts state changes; DashboardWindow updates charts upon notification.
    - **Why Appropriate:** Allows View to automatically refresh when Model state changes.
    - **Difficulty Without It:** The user would have to manually refresh the page.
5. **Facade**
    - **Problem Addressed:** Interacting with an LLM requires complex prompt construction and JSON parsing.

- **Participating Classes:** AgentFacade, LLMClient, PromptBuilder.
- **Role of Each Class:** AgentFacade exposes a simple categorize() method, hiding the complexity of LLMClient and PromptBuilder.
- **Why Appropriate:** Shields the MainController from low level API orchestration.
- **Difficulty Without It:** UI controllers would be cluttered with HTTP request logic and string manipulation.

2.2 Use Case Diagram and Use Case Descriptions

- **Actors:** Primary User, External LLM Service.



UC01 Import Data

- **Use Case ID:** UC01
- **Use Case Name:** Import Data
- **Actor(s):** Primary User
- **Goal:** Load transactions into the application.
- **Preconditions:** User has a valid CSV file.
- **Trigger:** User clicks "Import".
- **Main Success Scenario:** System reads CSV, standardizes data, and saves to database.
- **Alternative/Exception Flows:** File is corrupted > System shows "Format Error" message.
- **Postconditions:** Database contains new uncategorized transactions.
- **Related Feature(s):** F01, F10.

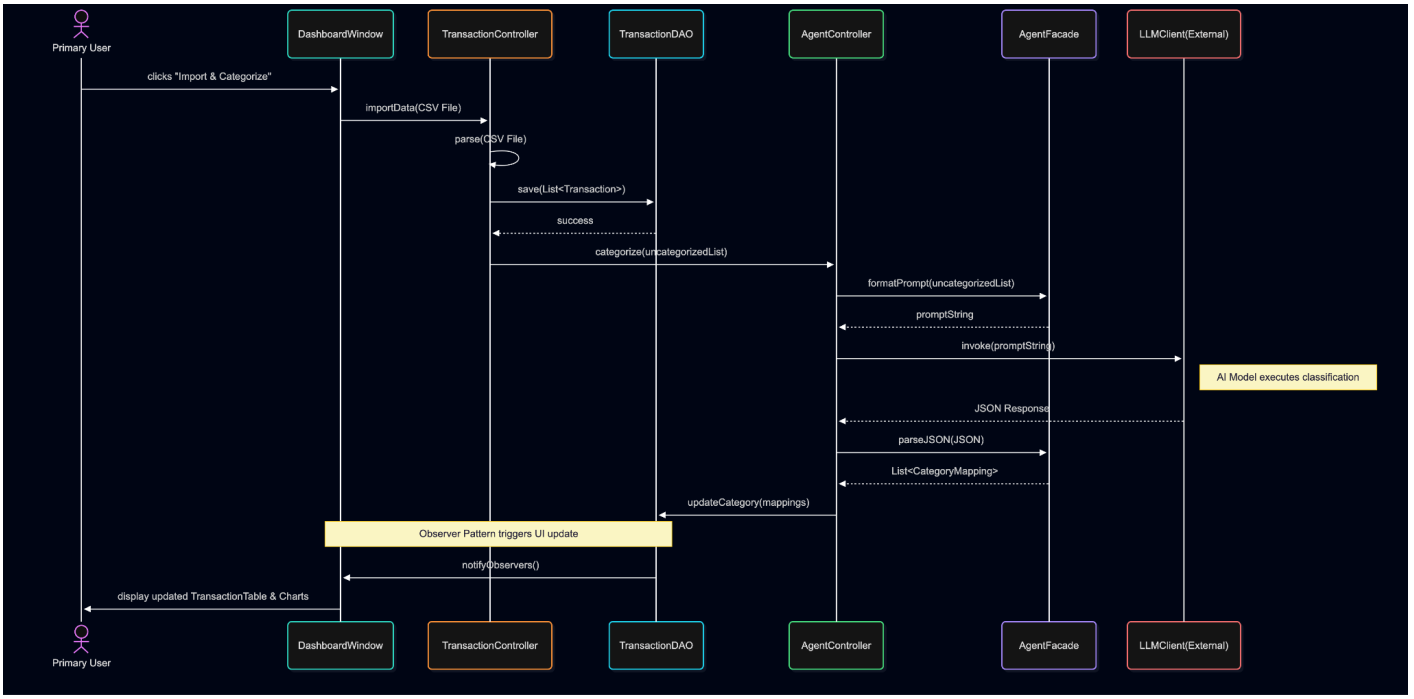
UC02 Categorize Transactions

- **Use Case ID:** UC02
- **Use Case Name:** Categorize Transactions
- **Actor(s):** Primary User, External LLM Service
- **Goal:** Assign categories to raw transactions.
- **Preconditions:** Database contains uncategorized transactions.
- **Trigger:** User clicks "Run AI Categorization".
- **Main Success Scenario:** System sends descriptions to LLM. LLM returns JSON categories. System updates database.
- **Alternative/Exception Flows:** API timeout > System categorizes what it can and pauses.
- **Postconditions:** Transactions have category labels.
- **Related Feature(s):** F02, F04, F05, F08.

UC03 Query Finances

- **Use Case ID:** UC03
- **Use Case Name:** Query Finances
- **Actor(s):** Primary User, External LLM Service
- **Goal:** Gain insight via chat.
- **Preconditions:** Categorized data exists.
- **Trigger:** User sends a chat message.
- **Main Success Scenario:** Agent parses query, executes SQL tool, formats results, and returns natural language answer.
- **Alternative/Exception Flows:** Unrelated prompt > AI reminds user to ask about finances.
- **Postconditions:** User views answer in chat window.
- **Related Feature(s):** F03.

2.3 Sequence Diagrams



SD01 Import and Categorize

- **Initiating Actor:** Primary User
- **Flow:**
    1. Primary User clicks "Import & Categorize" on DashboardWindow (GUI/Boundary).
    2. DashboardWindow calls importData(file) on TransactionController.
    3. TransactionController calls save() on TransactionDAO (Domain Object).
    4. TransactionController calls categorize() on AgentController.
    5. AgentController calls invoke() on LLMClient (AI Component).
    6. LLMClient calls External LLM Service and receives JSON.
    7. AgentController calls updateCategories() on TransactionDAO.
    8. TransactionDAO notifies DashboardWindow of update.

SD02 Natural Language Query

- **Initiating Actor:** Primary User
- **Flow:**
    1. Primary User submits text to ChatPanel.
    2. ChatPanel calls processQuery() on AgentController.
    3. AgentController uses ToolManager to call executeSQL() on TransactionDAO.
    4. TransactionDAO returns data to AgentController.
    5. AgentController passes context to LLMClient via invoke().
    6. LLMClient returns text response to AgentController.
    7. AgentController returns text to ChatPanel for display.

Task 3: Feature to Design Traceability

Feature	Description	Type	Related Use Case	Classes	Key Methods	Sequence Diagram	Design Pattern(s)
F01	CSV Import	Deterministic	UC01	TransactionController, ImportStrategy, TransactionDAO	importData(), parse(), save()	SD01	Strategy, MVC
F02	AI Categorize	AI	UC02	AgentController, AgentFacade, LLMClient	categorize(), invoke(), parseJSON()	SD01	Facade, Singleton
F03	NL Queries	AI	UC03	ChatPanel, QueryAgent, ToolManager	sendQuery(), executeSQL(), invoke()	SD02	MVC
F04	Anomaly Detect	Hybrid	UC02	MainController, TransactionDAO, AgentController	getAverages(), flagAnomaly()	SD01	Observer
F05	Recurring	Hybrid	UC02	TransactionController, TransactionDAO	findRecurring()	SD01	MVC
F06	Budgeting	Deterministic	UC04	DashboardWindow, Budget, TransactionDAO	setLimit(), getProgress()	None	Observer

F07	Summary Gen	AI	UC05	AgentController, LLMClient, Report	generateSummary(), invoke()	SD02	Facade
F08	Manual Override	Hybrid	UC02	TransactionTable, TransactionDAO, CategorizationAgent	updateCategory(), updateMemory()	None	Observer
F09	Report Export	Deterministic	UC05	MainController, Report, TransactionDAO	exportData(), formatCSV()	None	MVC
F10	CLI Engine	Hybrid	UC01, UC02	CLIParser, TransactionController, AgentController	parseArgs(), runBatch()	SD01	MVC

Task 4: Feature Explanation

- **F01 (CSV Import):** The DashboardWindow or CLIParser passes a file to the TransactionController. The controller uses an ImportStrategy to extract data, instantiates Transaction objects, and persists them via the TransactionDAO.

- **F02 (AI Categorization):** The AgentController retrieves uncategorized objects from the TransactionDAO. It passes these to the AgentFacade, which formats a prompt and calls the LLMClient. The returned JSON updates the database.

- **F03 (NL Queries):** The ChatPanel captures user text. QueryAgent analyzes the intent, uses ToolManager to pull relevant database aggregates from TransactionDAO, and passes the context to the LLMClient to format a conversational reply.

- **F04 (Anomaly Detection):** Following a database update, MainController calculates rolling averages. If a threshold is broken, it triggers the AgentController to draft a warning alert displayed on the GUI.

- **F05 (Recurring Detection):** A process in TransactionController scans the TransactionDAO for matching vendor amount pairs spaced ~30 days apart, tagging them in the database.

- **F06 (Monthly Budget Creation):** User sets integers in the DashboardWindow. The Controller maps these to Budget objects. When the TransactionDAO updates, the Observer pattern notifies the UI to recalculate progress bars.

- **F07 (Summary Gen.):** Triggered via UI, the AgentController gathers monthly Category totals from TransactionDAO and invokes the LLMClient to write a comprehensive text summary.

- **F08 (Manual Override):** User alters a dropdown in TransactionTable. The Controller updates TransactionDAO. It also sends the new pair to the CategorizationAgent's local memory to prevent future misclassifications.

- **F09 (Report Export):** MainController pulls filtered data from TransactionDAO, formats it into a Report object, and writes it to the local file system.

- **F10 (CLI Batch Processing Engine):** CLIParser intercepts terminal execution, bypassing the DashboardWindow, and directly invokes the necessary methods on TransactionController and AgentController to execute workflows headlessly.